

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

FACULTY: Dr. D.SUDHAGAR

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I Haresh Kumar N L (192425009) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: haresh

QUESTIONS: 05

Total Marks:25

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

- Identify the conceptual error in the likelihood formulation.
- Explain how this mistake affects parameter estimation.
- Suggest the correct mathematical formulation and reasoning.
- How would using log-likelihood fix computational issues?

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

- a) List possible implementation errors in the learning algorithm.
- b) Analyze how incorrect learning rate or weight updates affect convergence.
- c) Identify dataset-related issues that may cause this problem.
- d) Propose modifications to ensure convergence.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

- a) Identify possible mathematical or coding errors in gradient computation.
- b) Explain the role of activation functions in gradient flow.
- c) Analyze how vanishing or exploding gradients affect training.
- d) Suggest techniques to fix the issue (e.g., normalization, initialization).

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

- a) Explain why batch gradient descent is slow in large datasets.
- b) Analyze the cause of fluctuations in SGD.
- c) Compare the error behavior of Mini-Batch Gradient Descent.
- d) Recommend the best approach and justify with error trade-offs.

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

- a) Explain how high dimensionality increases model error.
- b) Identify signs of overfitting in this scenario.
- c) Analyze why distance-based learning fails in high dimensions.
- d) Suggest dimensionality reduction or regularization techniques to improve performance.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Answer

1)

a) Identify the conceptual error in the likelihood formulation.

The error is that the likelihood is written as the **sum of probabilities** instead of the **product of probabilities**.

Incorrect formulation:

$$L(\theta) = \sum_{i=1}^n P(x_i | \theta)$$

Correct formulation:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

Since observations are assumed to be **independent**, the joint probability is the **product** of individual probabilities.

b) Explain how this mistake affects parameter estimation.

- The optimization objective becomes mathematically incorrect.
- Estimated parameters no longer maximize the joint probability of the observed data.
- Even with a large dataset, the model produces biased or inaccurate estimates.
- Classification accuracy decreases because the model does not learn the true data distribution.

c) Suggest the correct mathematical formulation and reasoning.

For independent observations:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

The Maximum Likelihood Estimator is

$$\hat{\theta} = \arg \max_{\theta} L(\theta)$$

This formulation correctly represents the probability of observing the entire dataset.

d) How would using log-likelihood fix computational issues?

Taking logarithms converts the product into a sum:

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

Advantages:

- Prevents numerical underflow.
- Simplifies differentiation.
- Makes optimization computationally efficient.
- Produces the same optimal parameter estimates because the logarithm is a monotonic function.

2)

a) List possible implementation errors in the learning algorithm.

Possible errors include:

- Incorrect weight update rule.
- Wrong bias update.
- Incorrect prediction function.
- Wrong class labels (using 0/1 instead of -1/+1 where required).
- Missing learning rate.
- Incorrect stopping condition.

b) Analyze how incorrect learning rate or weight updates affect convergence.

- **Very large learning rate:** Causes overshooting and oscillations.
- **Very small learning rate:** Leads to slow convergence.
- **Incorrect update equation:** Prevents the algorithm from moving toward the correct decision boundary.

Correct update rule:

$$w_{new} = w_{old} + \eta(y - \hat{y})x$$

where:

- η = learning rate
- y = true label
- $\hat{y}$ = predicted label

c) Identify dataset-related issues that may cause this problem.

Possible issues:

- Incorrect data labels.
- Duplicate or noisy samples.

- Features not properly scaled.
- Dataset is actually not linearly separable.
- Missing or corrupted training examples.

d) Propose modifications to ensure convergence.

- Normalize input features.
- Use the correct weight update rule.
- Select an appropriate learning rate.
- Verify labels and preprocessing.
- Shuffle training data before each epoch.
- Confirm that the dataset is linearly separable.

3)

a) Identify possible mathematical or coding errors in gradient computation.

Possible errors:

- Incorrect derivative of activation function.
- Wrong chain rule implementation.
- Incorrect sign in gradient update.
- Updating weights before computing all gradients.
- Incorrect learning rate.
- Programming bugs in matrix operations.

b) Explain the role of activation functions in gradient flow.

Activation functions introduce non-linearity.

During backpropagation, gradients depend on their derivatives.

Examples:

- Sigmoid derivative:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

- ReLU derivative:

$$f'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

If derivatives become very small, gradients diminish, slowing learning.

c) Analyze how vanishing or exploding gradients affect training.

Vanishing gradients

- Gradients approach zero.
- Early layers learn extremely slowly.
- Training may stop improving.

Exploding gradients

- Gradients become excessively large.
- Weight updates become unstable.
- Loss increases rapidly or becomes NaN.

d) Suggest techniques to fix the issue.

Recommended techniques:

- Xavier or He weight initialization.
- Batch Normalization.
- Gradient Clipping.
- Use ReLU instead of sigmoid in hidden layers.
- Normalize input data.
- Choose an appropriate learning rate.
- Use adaptive optimizers such as Adam or RMSProp.

4)

a) Explain why batch Gradient Descent is slow in large datasets.

Batch Gradient Descent computes gradients using the **entire dataset** before each update.

Therefore:

- High computational cost.
- Large memory usage.
- Slow parameter updates.
- Longer training time for large datasets.

b) Analyze the cause of fluctuations in SGD.

SGD updates parameters using one sample at a time.

Therefore:

- Gradient estimates are noisy.
- Loss fluctuates during training.
- Faster updates but less stable convergence.

c) Compare the error behavior of Mini-Batch Gradient Descent.

Mini-Batch Gradient Descent uses a small subset of data.

Comparison:

Method	Error Behavior
Batch GD	Smooth but slow convergence
SGD	Fast but highly fluctuating
Mini-Batch GD	Moderate speed with reduced fluctuations

Mini-batch provides a balance between stability and computational efficiency.

d) Recommend the best approach and justify with error trade-offs.

Mini-Batch Gradient Descent is generally the best choice because it:

- Converges faster than Batch GD.
- Produces less noisy updates than SGD.
- Requires moderate memory.
- Is well suited for modern deep learning.

Thus, it offers the best trade-off between speed, stability, and generalization.

5)

a) Explain how high dimensionality increases model error.

As the number of features increases:

- Data becomes sparse.
- More training samples are required.
- The model learns noise instead of patterns.
- Variance increases, leading to overfitting.

b) Identify signs of overfitting in this scenario.

Common signs include:

- Very high training accuracy.
- Low test accuracy.
- Large gap between training and testing performance.
- Poor generalization to unseen data.

c) Analyze why distance-based learning fails in high dimensions.

In high-dimensional spaces:

- Distances between data points become very similar.
- The concept of "nearest neighbor" loses significance.
- Similarity measures become less informative.
- Algorithms such as KNN and K-Means perform poorly because distance no longer effectively distinguishes samples.

d) Suggest dimensionality reduction or regularization techniques to improve performance.

Dimensionality Reduction

- Principal Component Analysis (PCA)
- Linear Discriminant Analysis (LDA)
- Autoencoders
- Feature Selection (Chi-Square, Mutual Information)

Regularization

- L1 Regularization (LASSO)
- L2 Regularization (Ridge)
- Elastic Net
- Dropout (for neural networks)
- Early Stopping

These techniques reduce model complexity, remove irrelevant features, minimize overfitting, and improve test performance and generalization.